

Lightning lesson, sixty minutes

Automate Pull Requests with Background Coding Agents

A live run from an untouched issue to a reviewable pull request. The demonstration is the session; commentary fits around it.

Thirty minutes ahead

Reset the board

Goal The scratch repository is back to its seed, with the three prepared items open and none of them touched.

Say Nothing. This is the riskiest preparation in the series, so leave time for it.

Run

```
$ make lab-reset
$ make demo STEP=06 -- --dry-run
```

Check

- Three open items and no pull requests.
- The dry run plans all three and writes nothing.
- A second terminal ready, because the run and the console are watched together.

Ask *Nobody. Decide now what you will do if the network drops, and keep the recording as the fallback.*

0:00 to 0:12

An issue an agent can act on

Goal Attendees can tell the difference between an item that is ready and one that only looks ready.

- Say**
- Read the three prepared items aloud and ask the room to sort them before the machine does.
 - Draw out what makes the difference: a checkable finishing condition, the files it is expected to touch, and whether the intent needs a person.
 - Point out that triage is a separate stage for exactly this reason. Merging it into implementation is how a narrow request becomes a wide change.
-

Run

```
$ node steps/06-delivery/demo.ts triage
```

- Room sees**
- Each item labelled, with the reason recorded beside it.
 - One sent to a person, and the question it needs answered.
-

Ask *Open your own backlog. How many of the top five could a machine start on today?*

0:12 to 0:36

Let it work

Goal	The room watches a change get made and checked without anyone steering it.
Say	• Start it, then stop talking for a moment so the room registers that nobody is driving. • Narrate each stage as it appears and keep naming which layer is responsible. • Use the waiting time for the question the room always asks, which is what happens when it gets something wrong. The answer arrives in the next segment.
Run	\$ make demo STEP=06
Room sees	• The item claimed and a worktree appearing. • Context assembled, then a change, then three suites running in three languages. • Both verdicts, then a draft pull request opening.
Ask	<i>While this runs: what would you need to see attached before you would read one of these?</i>

0:36 to 0:46

The one it will not do

Goal The room sees a refusal treated as a successful outcome rather than an error.

- Say**
- Set it up before running it. This item needs the pricing rules, and pricing decides what a customer pays.
 - Run it and let the stop sit on screen without narration for a second.
 - Explain what would have happened without the rule: a plausible change to money, verified by tests the same run wrote.
-

Run

```
$ node steps/06-delivery/demo.ts refused
```

- Room sees**
- The run stopping before any file is written.
 - The rule that stopped it, quoted from the charter, and the item routed to a person.
-

Ask *Which directory in your repository would you put on that list first?*

0:46 to 0:56

What arrives for a person

Goal They see that the decision is unchanged and only the preparation for it is different.

Say

- Open the pull request and read its evidence section top to bottom without editorialising.
- Say the sentence the room needs to hear: the reviewer is not removed, they arrive after everything a machine can check is done.
- Show the console and let the stage timings make the point about where time goes.

Run

```
$ node steps/06-delivery/demo.ts open-pr
```

Room sees

- A draft pull request with both verdicts, the negative proof and the cost attached.
- The console listing the run, with its stages as a waterfall.

Ask *Is that enough to review in five minutes? What is missing for you?*

0:56 to 1:00

Where the ceiling is

Goal	Nobody leaves believing more agents is the next move.
Say	• Name the real limit: how many decisions can be waiting on one person, not how much can be generated. • Mention that several verified requests on one base branch still have to be integrated by somebody.
Run	Leave the pull request list on screen.
Room sees	More than one request awaiting review, and the production stopping.
Ask	<i>How many pending decisions can you genuinely absorb in a week?</i>

0:35 to 0:45

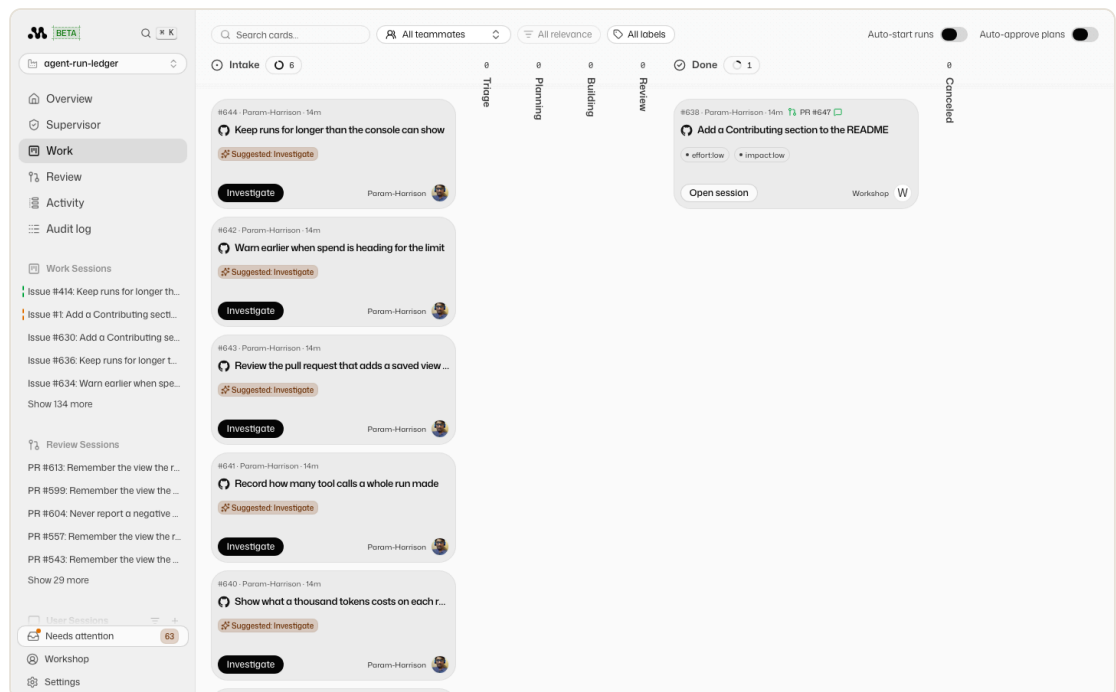
The pull request as the interface

Goal They can describe the hand-off point between a machine and a person in one sentence.

- Say**
- Every route in this material ends at a draft, on every tier, without exception.
 - The pull request is where the evidence lives, which is what makes it reviewable by someone who watched none of the run.
 - Contrast that with a chat transcript, which nobody can audit six weeks later.

Run

```
$ gh pr list --repo learnwithparam/agent-run-ledger
```



The change, its evidence, and nothing merged.

Show on the board

Review to the pull request to Files changed to the plan artefact

The evidence travels with the change

The plan the run wrote is committed beside the diff, so a reviewer who watched none of it can see what was intended before reading what was done.

Nothing merges itself, on any tier

Every route in this material ends at an open pull request. The charter says so and the four shapes do not argue about it.

A decision is not the work

The board records a decision succeeding within seconds while the agent goes on working for minutes. Anyone timing a stage from the decision record will report a factory four times faster than it is.

Room does

- Opens the pull request and finds the commands the run actually executed.
- Confirms nothing was merged by anything except a person.

Ask

What in this pull request would you need if you were reviewing it a month from now?

Concepts: the queue, a request with its evidence, the merge gate, the run record.